

```
1 import java.util.Comparator;
11
12 /**
13  * Reads from a words and definitions file and creates an organized
14  * html index
15  * page with each word listed alphabetically and linked to its own
16  * html page
17  * that contains the word's definition and a link back to the index
18  * page.
19  *
20  * @author Connor Asari
21  */
22 //I need to keep this public so I can write test cases for methods.
23 public final class Glossary {
24     /**
25      * The Comparator that sorts 2 Strings alphabetically.
26      */
27     private static final class StringLT implements
28     Comparator<String> {
29         @Override
30         public int compare(String str1, String str2) {
31             return str1.compareTo(str2);
32         }
33     }
34     /**
35      * No argument constructor--private to prevent instantiation.
36      */
37     public Glossary() {
38     }
39
40     /**
41      * Reads thru an entire input file of words and definitions and
42      * adds each
43      * word and corresponding definition to a Map<String word,
44      * String
45      * definition>.
46      *
47      * @param in
48      *         the input stream
49      *
50      * @return the Map full of words and definitions
```

```
49      *
50      * @updates in.content
51      *
52      * @requires in is initialized to read from a file formatted
53      *           with 1 word on
54      *           the first line, followed by its definition of 1 or
55      *           more lines,
56      *           followed by a blank line, followed 0 or more
57      *           word+definition
58      *           pairs (of which must also be in the previously
59      *           stated format)
60      *
61      * @ensures the Map contains all word+definition pairs from the
62      *           file, and
63      *           each word corresponds with its correct definition,
64      *           as formatted
65      *           in the file.
66      */
67      public static Map<String, String> getWordsAndDefs(SimpleReader
68      in) {
69          Map<String, String> result = new Map1L<String, String>();
70
71          while (!in.atEOS()) {
72              //The first line is always the word
73              String word = in.nextLine();
74              //The next line is always the 1st line of the
75              definition
76              //There might be more lines of the definition
77              String def = in.nextLine();
78
79              //If the file is not at end of stream, read in the next
80              line
81              String nextLine = "";
82              if (!in.atEOS()) {
83                  nextLine = in.nextLine();
84              }
85
86              //While the next line isn't empty, the definition is
87              continuing,
88              //so keep taking the next line.
89              while (!nextLine.equals("") && !in.atEOS()) {
90                  def = def.concat(" " + nextLine);
91                  nextLine = in.nextLine();
92              }
93              result.add(word, def); //Add the word and definition to
```

```
the Map
84
85     }
86     return result;
87 }
88
89 /**
90  * Sorts all Strings in the Queue alphabetically starting at
the first
91  * letter.
92  *
93  * @param wordsAndDefs
94  *         the Map<K,V> where each K is the word to be
sorted and added
95  *         into the Queue
96  * @return the Queue of alphabetically sorted words
97  *
98  * @requires each K in the Map<K,V> contains at least 1
character.
99  *
100  * @ensures the Queue is sorted in alphabetical order, where
the first word
101  *         alphabetically is at the front of the Queue.
102  */
103 public static Queue<String> sortWordsAlphabetically(
104     Map<String, String> wordsAndDefs) {
105
106     Queue<String> result = new Queue1L<String>(); //Empty Queue
107
108     //For each word+definition pair in the Map,
109     for (Map.Pair<String, String> pair : wordsAndDefs) {
110         //Add the word to the Queue
111         result.enqueue(pair.key());
112     }
113     //Now I have a Queue full of all the words from the Map
114     //Construct a Comparator to sort the Queue
115     Comparator<String> alphaOrder = new StringLT();
116
117     result.sort(alphaOrder); //Sort the Queue and return it
118     return result;
119 }
120
121 /**
122  * Generates an index file with a list of each word from the
Queue, as well
```

```
123     * as a hyperlink to a separate html page with that word's
124     defintion.
125     * @param words
126     *         the Queue of words to be added to the glossary,
127     starting with
128     *         the words at the front of the Queue.
129     * @param wordsAndDefs
130     *         the Map<K,V> where each K is a word, and each V
131     is that word's
132     *         corresponding definition.
133     * @param folderName
134     *         the name of the folder in which to write the html
135     file into.
136     * @clears words
137     *
138     * @requires words != {} && the folder (folderName) exists
139     within the
140     *         project
141     * @ensures ???
142     */
143     private static void generateIndexFile(Queue<String> words,
144         Map<String, String> wordsAndDefs, String folderName) {
145
146         //Initialize output stream to write to the file
147         SimpleWriter out = new SimpleWriter1L(folderName + "/"
148     index.html");
149
150         out.println("<!DOCTYPE html>");
151         out.println("<html lang=\"en\">");
152
153         //Write the page title
154         out.println("    <head>");
155         out.println("        <title>Connor's Glossary</title>");
156         out.println("    </head>\n");
157
158         out.println("    <body>");
159
160         //Write the header
161         out.print("        <h1>");
162         out.print("Connor's Glossary");
```

```
162         out.println("</h1>");
163
164         //Write the other header
165         out.print("    <h3>");
166         out.print("Index of words:");
167         out.println("</h3>");
168
169         //Write the list of words and generate an html file for
each word
170         out.println("    <ul>");
171         while (words.length() > 0) {
172             out.print("        <li>");
173
174             String word = words.dequeue(); //Take the first word
from the queue
175             //Generate an html file for that word
176             generateWordFile(word, wordsAndDefs, folderName);
177             //Print the word with a link to its definition page
178             String url = word + ".html";
179             out.print("<a href=\"\" + url + \"\">\" + word + "</a>");
180
181             out.println("</li>");
182         }
183         out.println("    </ul>");
184
185         out.println(" </body>");
186
187         out.println("</html>");
188         out.close(); //Close the output stream
189     }
190
191     /**
192     * Generates an html file with the word as the title, followed
by its
193     * definition, followed by a link to return to the index page.
If another
194     * word from the glossary appears, its html page is linked as
well.
195     *
196     * @param word
197     *         the word of which to display the definition of.
198     *
199     * @param wordsAndDefs
200     *         the Map<K,V> where each K is a word, and each V
is its
```

```
201      *          definition.
202      *
203      * @param folderName
204      *          the name of the folder in which to write the html
        file into.
205      *
206      * @requires word is a key within the Map, the folder
        folderName exists
207      *          within the project.
208      *
209      * @ensures The html file generated contains word in red bold
        italicized
210      *          font, the displayed definition matches word, the
        return to index
211      *          link properly sends the user back the index page.
212      */
213      private static void generateWordFile(String word, Map<String,
        String> wordsAndDefs,
214          String folderName) {
215
216          //Initialize output stream to write to the file
217          SimpleWriter out = new SimpleWriter1L(folderName + "/" +
        word + ".html");
218
219          out.println("<!DOCTYPE html>");
220          out.println("<html lang=\"en\">");
221
222          //Write the page title
223          out.println("  <head>");
224          out.println("    <title>" + word + "</title>");
225          out.println("  </head>\n");
226
227          out.println("  <body>");
228
229          //Write the word in red bold italics
230          out.println(
231              "    <h1 style=\"color: red; font-weight: bold;
        font-style: italic;\n">");
232          out.println("      " + word);
233          out.println("</h1>");
234
235          //Write the definition
236          out.print("    <p>          ");
237          printDefinition(word, wordsAndDefs, out);
238          out.println("</p>");
```

```
239
240     //Link back to the index page
241     out.print("    <p>");
242     out.print("Return to ");
243     out.print("<a href=\"index.html\">Index</a>");
244     out.println("</p>");
245
246     out.println("  </body>");
247
248     out.println("</html>");
249     out.close();
250 }
251
252 /**
253  * Prints the definition that corresponds to (@code word), if
  any word
254  * within the definition also appears as a K within the
  Map<K,V>, link that
255  * word to its separate term page.
256  *
257  * @param word
258  *         the word of whose definition to print.
259  *
260  * @param wordsAndDefs
261  *         the Map<K,V> of words and definitions to
  reference.
262  *
263  * @param out
264  *         output stream.
265  *
266  * @updates out.content
267  *
268  * @requires (@code word) is a key within (@code wordsAndDefs).
269  *
270  * @ensures the definition that corresponds to (@code word) as
  it exists in
271  *         (@code wordsAndDefs) is printed, and every word
  within that
272  *         definition that exists as its own key within (@code
  wordsAndDefs) is linked to its own html page.
273  */
274
275 private static void printDefinition(String word, Map<String,
  String> wordsAndDefs,
276     SimpleWriter out) {
277     String fullDef = wordsAndDefs.value(word); //Get the full
```

```

    definition
278
279     while (fullDef.length() > 0) {
280         //Get the next string of consecutive letters or string
    of consecutive
281         //non-letters
282         String next = nextWordOrSeparator(fullDef);
283
284         //If the word appears as a K within the Map<K,V>
285         if (wordsAndDefs.containsKey(next)) {
286             //Print the word as a link to its individual html
    page
287             String url = next + ".html";
288             out.print("<a href=\"" + url + "\">" + next + "</
a>");
289         } else {
290             //If the word doesn't appear in the Map, print it
    normally
291             out.print(next);
292         }
293         //Reduce the fullDef to only the un-read portion
294         fullDef = fullDef.substring(next.length());
295     }
296
297 }
298
299 /**
300  * Returns the first "word" (maximal length string of
    characters not in
301  * {@code separators}) or "separator string" (maximal length
    string of
302  * characters in {@code separators}) in the given {@code text}
    starting at
303  * the given {@code position}.
304  *
305  * @param text
306  *     the {@code String} from which to get the word or
    separator
307  *     string
308  *
309  * @return the first word or separator string found in {@code
    text} starting
310  *     at index {@code position}
311  * @requires 0 <= position < |text|
312  * @ensures <pre>

```



```
313     * nextWordOrSeparator =
314     *   text[position, position + |nextWordOrSeparator|) and
315     *   if entries(text[position, position + 1)) intersection
       separators = {}
316     * then
317     *   entries(nextWordOrSeparator) intersection separators = {}
and
318     *   (position + |nextWordOrSeparator| = |text| or
319     *   entries(text[position, position + |nextWordOrSeparator| +
       1))
320     *   intersection separators /= {})
321     * else
322     *   entries(nextWordOrSeparator) is subset of separators and
323     *   (position + |nextWordOrSeparator| = |text| or
324     *   entries(text[position, position + |nextWordOrSeparator| +
       1))
325     *   is not subset of separators)
326     * </pre>
327     */
328     public static String nextWordOrSeparator(String text) {
329
330         String result = "";
331         if (text.length() > 0) {
332             boolean startsWithSeparator = !
Character.isLetter(text.charAt(0));
333             int endIndex = text.length();
334             if (!startsWithSeparator) {
335                 int i = 1;
336                 boolean foundSeparator = false;
337                 while (!foundSeparator && i < text.length()) {
338                     if (!Character.isLetter(text.charAt(i))) {
339                         foundSeparator = true;
340                         endIndex = i;
341                     } else {
342                         i++;
343                     }
344                 }
345             } else if (startsWithSeparator) {
346                 int i = 1;
347                 boolean foundLetter = false;
348                 while (!foundLetter && i < text.length()) {
349                     if (Character.isLetter(text.charAt(i))) {
350                         foundLetter = true;
351                         endIndex = i;
352                     } else {
```

```
353             i++;
354         }
355     }
356 }
357     result = text.substring(0, endIndex);
358 }
359
360     return result;
361 }
362
363 /**
364  * Main method.
365  *
366  * @param args
367  *     the command line arguments
368  */
369 public static void main(String[] args) {
370     SimpleReader in = new SimpleReader1L();
371     SimpleWriter out = new SimpleWriter1L();
372
373     out.println("Enter the name of a file: ");
374     String fileName = in.nextLine();
375     SimpleReader fileReader = new SimpleReader1L(fileName);
376
377     out.println("Enter the name of a folder to save output
378 files: ");
379     String folderName = in.nextLine();
380
381     //Get the Map of words and definitions, then sort the words
382     //alphabetically
383     Map<String, String> wordsAndDefs =
384         getWordsAndDefs(fileReader);
385     Queue<String> sortedWords =
386         sortWordsAlphabetically(wordsAndDefs);
387
388     //Create the index file (and subsequently all the
389     //definition files)
390     generateIndexFile(sortedWords, wordsAndDefs, folderName);
391
392     in.close();
393     out.close();
394 }
```